Mimari Görünümler

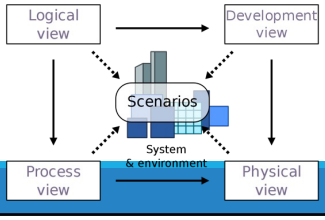
Mimari görünümler

Bina mimarları eskiz ve taslaklar ile önerilen bir binanın mimarisini ifade ederler.

Yazılım mimarları da görünüm (view) adı verilen gösterim araçlarını kullanır.

Mimari bir diyagram DEĞİLdir. Sistemin çözeceği problemi nasıl çözeceği için önemli tasarım kararları kümesidir.

- Gereksinimlerde yer alan problem için belirli bir çözümü, işlevsel olmayan gereksinimleri de karşılayacak şekilde hesaplamak için bir araya gelen bileşen kümesini ifade eder.
- Grafik ve metin türünde mimari görünümleri nasıl ifade edebildiğimize bakacağız.
- Kruchten'in makalesini baz alacağız. Sonrasında farklı görünümleri de (özellik, işlevsel olmayan gereksinimler, hata raporlama, bağlam, yardımcı program) görerek destekleyeceğiz.



Mantıksal Görünüm (Logical View)

İşlemsel, iletişimsel ve davranışsal sorumlulukların yapısal parçalanması

Diyagramlar:
- Kutu ve ok (Box and Arrow)
- UML sınıf modeli
- UML genel etkileşim
- UML İşbirliği
- Bileşenler ve Bağlayıcılar (ADL)

Mantıksal Görünüm: Kutu ve Ok

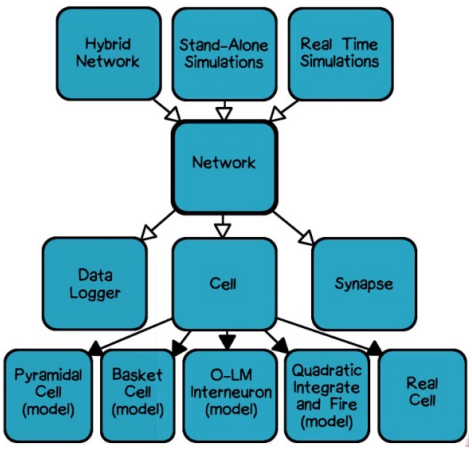
Kutular sistemin işlevsel bileşenleri

Oklar, aralarındaki bir çeşit bağımlılık ilişkisi

Tek diyagramda yüksek seviyeli mimari yapı

Anlaşılması kolay

Kesin olmayabilir.
- Oklar ne anlama geliyor?

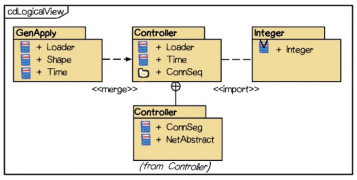


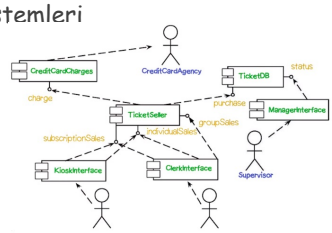
Gelişimsel Görünüm (Developmental View)

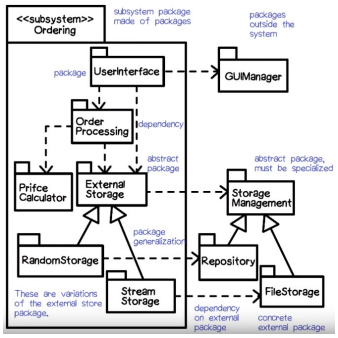
Burada kaynak kodu ile ilgiliyiz.

İlgilendiğimiz kaynak kodu birimleri: Paketler, sınıflar, altsistemler, kütüphaneler, dosyalar

Diyagramlar:
- UML Paket diyagramı
- UML Bileşen diyagramı
- CVS, SVN, Github vb kod kontrol sistemleri







Diyagramlardaki Bağlantılar

Bileşen Diyagramı: Çağrı, Kullanabilme

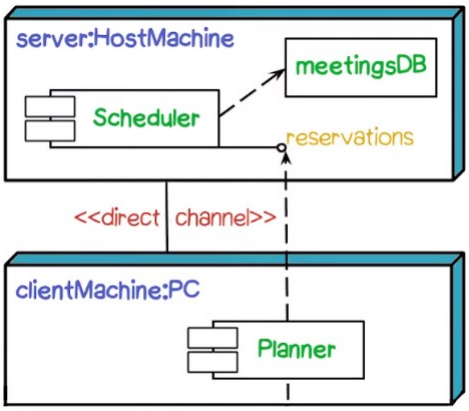
Paket Diyagramı: içeri aktarma (import), alt bileşen, bir araya gelme (merge)

Sınıf diyagramı: Genelleştirme, Bağımlılık

İşlem Görünümü (Process View)

Çalışmanın ayrıldığı eşzamanlı işlemler ve iş parçaları (processes & threads)

UML Barındırma (Deployment) Diyagramı



Fiziksel Görünüm (Physical View)

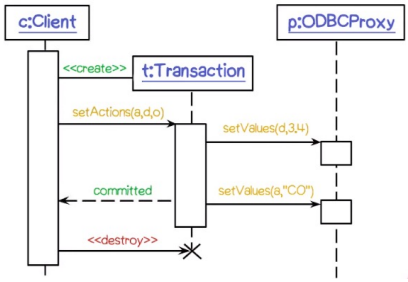
Bir öncekine çok yakın

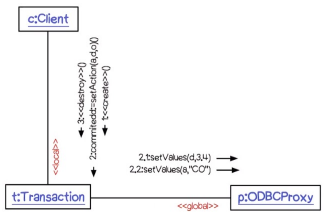
Sistemin çalışması için gerekli işletme birimleri (makineler) ve,

İşlemlerin çeşitli işletme birimlerine nasıl dağıldığı/yerleştirildiği

Diyagramlar:

- UML Barındırma Diyagramı
- UML Ardışıl (Sequence) veya UML İşbirliği (Collaboration) Diyagramı

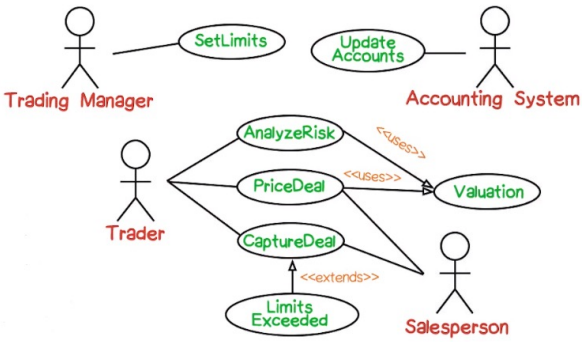




Kullanım Senaryosu Görünümü (Use Case View)

Harici aktörlerin gözünden/bakış açısından önemli işletim dizgeleri

Diyagramlar:
- UML Kullanım Senaryosu diyagramı
- UML İletişim (Communication) diyagramı
- UML Ardışıl diyagram
- UML Aktivite Diyagramı
- Bağlam Diyagramı (Taslak)
- Kullanım senaryoları: Metin ya da tablo olabilir.

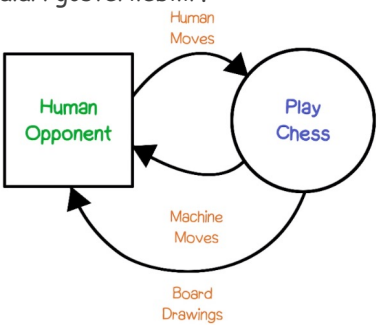


Bağlam Görünümü (Context View)

OMT'de yapısal ve davranışsal ve işlevsel görünüm vardı.

İşlevsel görünüm: Veri Akış Diyagramı (DFD)
- UML'de yok. Yerine Use-case diyagramı kullanılır.
- Sistemi tanımlarken oldukça faydalıdır. Aktiviteler ve oluşum sıraları gösterilebilir.

En üst seviyeli DFD bağlam diyagramı olarak da bilinir.
- Sistemin bulunduğu çevre ile olan etkileşimini vurgular.
- Harici aktörler/sistemler, veri akışları ile sisteme bağlıdır.
- Detaylar alt seviyeli (içiçe) diyagramlar ile gösterilir.



Özellik Görünümü (Feature View)

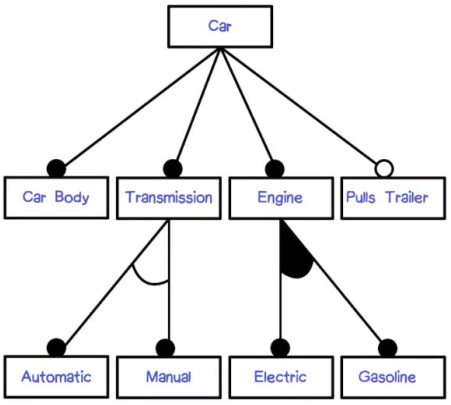
4+1 görünüm bitti ama sistemi mimari seviyede modellerken faydalı olabilecek başka görünümler de mevcuttur.

Özellik: Kullanıcının bakış açısından, kavramsal bir sistem davranışı

- Kamera: Zum özelliği (zum özelliği yok da olabilir.)

Özellik modellemesi: Birbiriyle ilişkili uygulamalar koleksiyonunun sağladığı bir özellik kümesini tanımlamak için kullanılır.

- Özellik diyagramı (Bir ürün kümesinde belirli bir ürüne uygulanabilecek özellikler)
  - Gerekli ve opsiyonel özellikler
  - Sadece biri seçilebilir / kombine edilebilir özellikler
- Özellikler arası kısıtlar
  - (diyagramda yok ama) Römork çekme sadece otomatik vites ile alınabilir)



Özellik Görünümü (Feature View)

Kaç çeşit araba oluşturulabilir?

Olasılıkları çarpmamız gerekir.

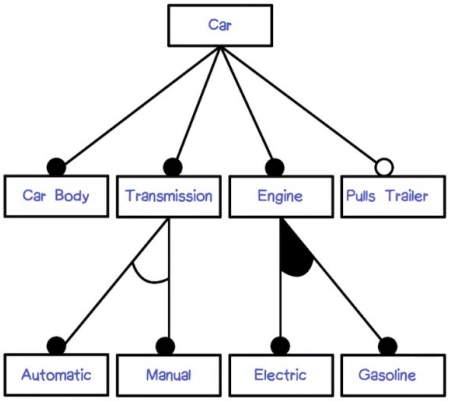
Gövde lazım ve alt özelliği yok.

Vites lazım ve 2 seçeneği var.

Motor lazım ve 3 seçeneği var (elektrik, benzin, elektrik+benzin)

İstersek römork çekebiliriz

1 x 2 x 3 x 2 = 12 çeşit araba



İşlevsel Olmayan Görünüm (Non Functional View)

Mimaride unutmamamız gereken husus: Sistem hesaplaması gerekeni, işlevsel olmayan gereksinimleri de karşılayacak şekilde hesaplamalıdır.

- Zordur ve birşeylerden ödün vermeyi gerektirir.
- Tablo yapısı kullanılabilir.

Bir web tarayıcısının esas sorumluluğu nedir?

- Web sayfalarını görüntülemek
- Ancak kaynak kodlarına bakarsak, «performans kısıtları için» pek çok önbellek yönetim mekanizması implementasyonu görürüz.
  - Sayfa önbelleği, bağlantı önbelleği, görüntü önbelleği, …
- Performans → Önbellek
- Güvenlik → SSL desteği
- Genişletilebilirlik → plug-in desteği
- Taşınabilirlik → cross-platform

Hata Raporlama Görünümü (Bug Reporting View)

Hataların (geliştirme ve kullanımda ortaya çıkan) yer aldığı birimler

Hata raporlama aracı bileşenleri

Hangi özelliği kullanırken ortaya çıktığı, geliştirmede kimin ilgilendiği de takip edilebilir.

Yardımcı Program Görünümü (Utility View)

Aslında tam bir görünüm değil ama diğer görünümler kadar yeterince üzerinde durulmayan sistem yapı özellikleri ile ilgili bilgiler içerir.

Sistemin bir parçası olabilecek ancak çalışan uygulamada yer almayan destek yazılımları.

- Kurulum betikleri (Installation scripts)
- Kayıt dosyası analizi (Log file analysis)
- İstatistiksel işleme
- Make dosyaları ve diğer kurulum betikleri
- Konfigürasyon dosyaları
- Belegelendirme (herhangi bir)
- Proje manifestoları (Geniş dağıtılabilir paketteki yapı tanımları)
- Destek araçları

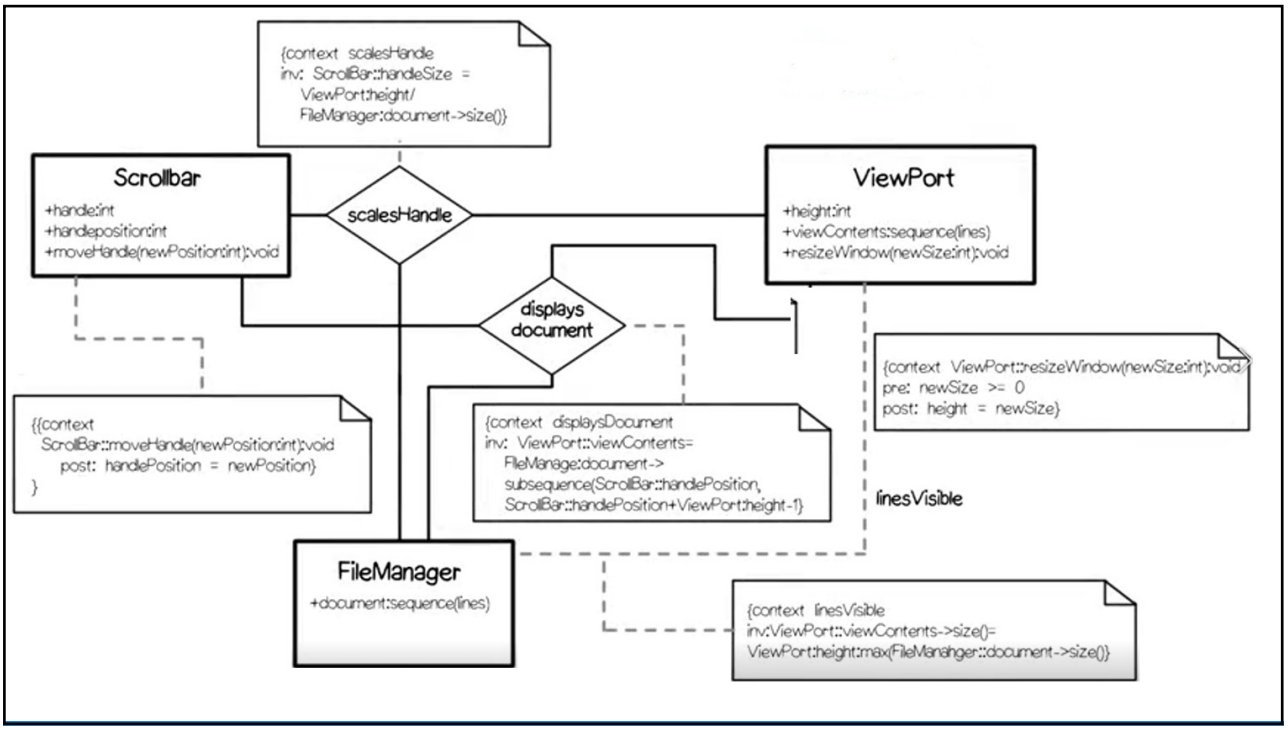
Özet

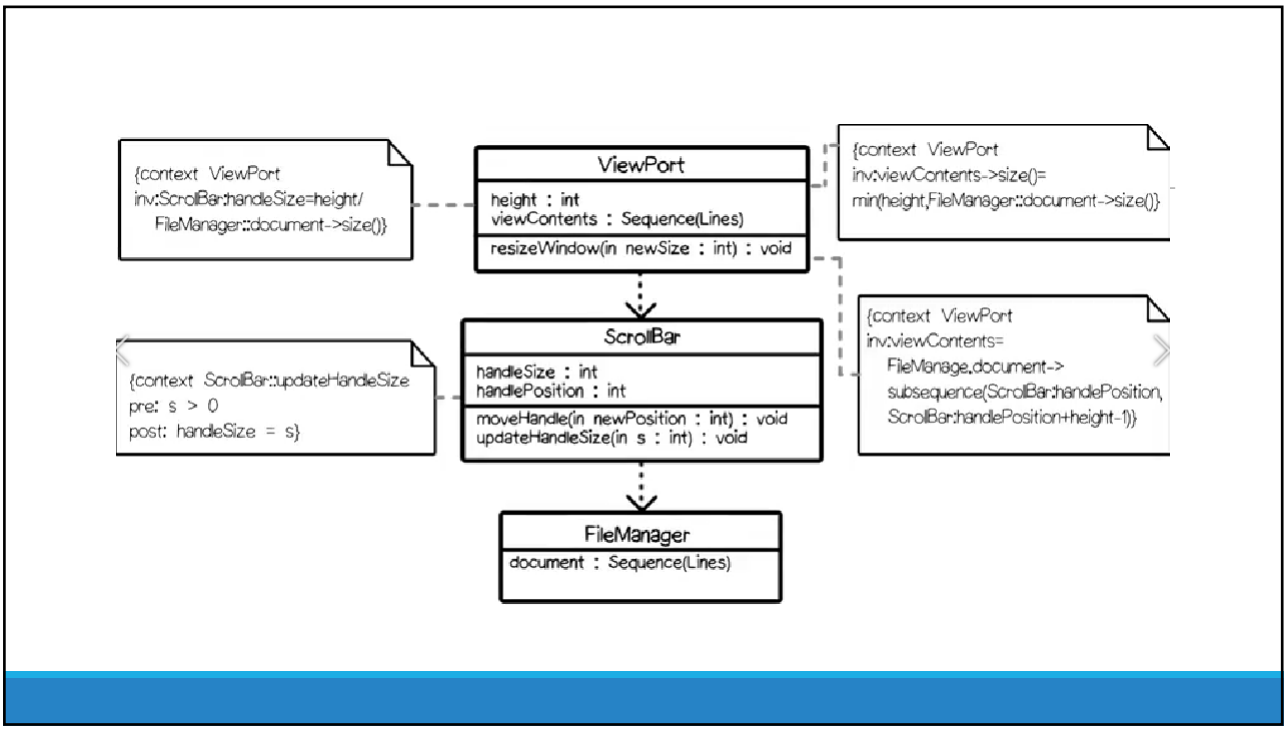
Mimaride somut ve tekil bir şey yoktur.

Mimari, bir karar kümesidir.

- Bu kararlar hakkındaki bilgiyi çeşitli (grafik/metin) görünümler üzerinden verebiliriz.
- Bunların toplamı mimariyi ifade eder.

Geliştirici olarak, sistemin yapısına, karmaşıklığına, uygulama alanına ve ilgililerine uygun görünümleri seçmemiz gerekir.


{context scalesHandle
inv: ScrollBar::handleSize =
ViewPort:height/
FileManager:document->size()}
Scrollbar
+handle:int
+handleposition:int
+moveHandle(newPosition:int):void
scalesHandle
ViewPort
+height:int
+viewContents:sequence(lines)
+resizeWindow(newSize:int):void
displays document
{context ViewPort::resizeWindow(newSize:int):void
pre: newSize >= 0
post: height = newSize}
{{context
ScrollBar::moveHandle(newPosition:int):void
post: handlePosition = newPosition}
}
{context displaysDocument
inv: ViewPort::viewContents=
FileManage:document->
subsequence(ScrollBar::handlePosition,
ScrollBar::handlePosition+ViewPort:height-1}
linesVisible
FileManager
+document:sequence(lines)
{context linesVisible
inv:ViewPort::viewContents->size()=
ViewPort:height:max(FileManahger::document->size()}


{context ViewPort
inv:ScrollBar:handleSize=height/
FileManager::document->size()}
ViewPort
height : int
viewContents : Sequence(Lines)
resizeWindow(in newSize : int) : void
{context ViewPort
inv:viewContents->size()=
min(height,FileManager::document->size()}
ScrollBar
handleSize : int
handlePosition : int
moveHandle(in newPosition : int) : void
updateHandleSize(in s : int) : void
{context ScrollBar::updateHandleSize
pre: s > 0
post: handleSize = s}
{context ViewPort
inv:viewContents=
FileManage.document->
subsequence(ScrollBar:handlePosition,
ScrollBar:handlePosition+height-1)}
FileManager
document : Sequence(Lines)